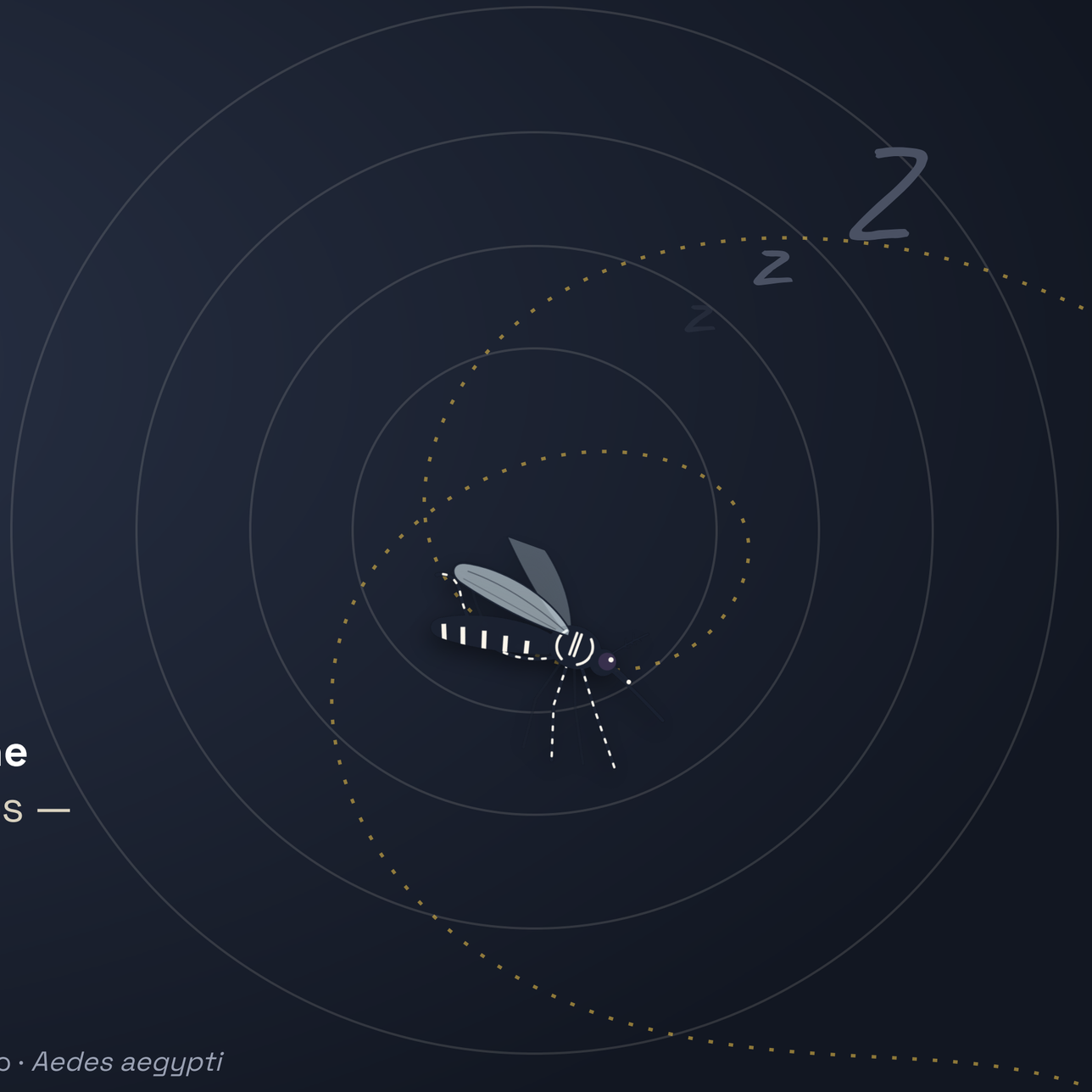
IFSC · IA na Eletrônica · 2026

Zzzz... é *Aedes*?



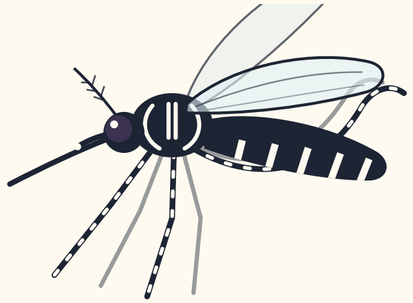
Como trabalha um desenvolvedor de **Machine Learning**: ferramentas, processo e resultados — ensinando uma máquina a *ouvir* mosquitos.

Luiz Eduardo do Nascimento Ciarallo

Projeto `mosquito-wingbeat` · detector ternário: ruído · outro mosquito · *Aedes aegypti*

o problema

Por que ouvir mosquitos?



Um vetor perigoso

O *Aedes aegypti* transmite **dengue, zika e chikungunya**. Saber **onde e quando** ele aparece orienta o combate.



Microfone barato

Uma armadilha com microfone pode monitorar **24 h por dia**, sem ninguém olhar lupa. O som vira dado.



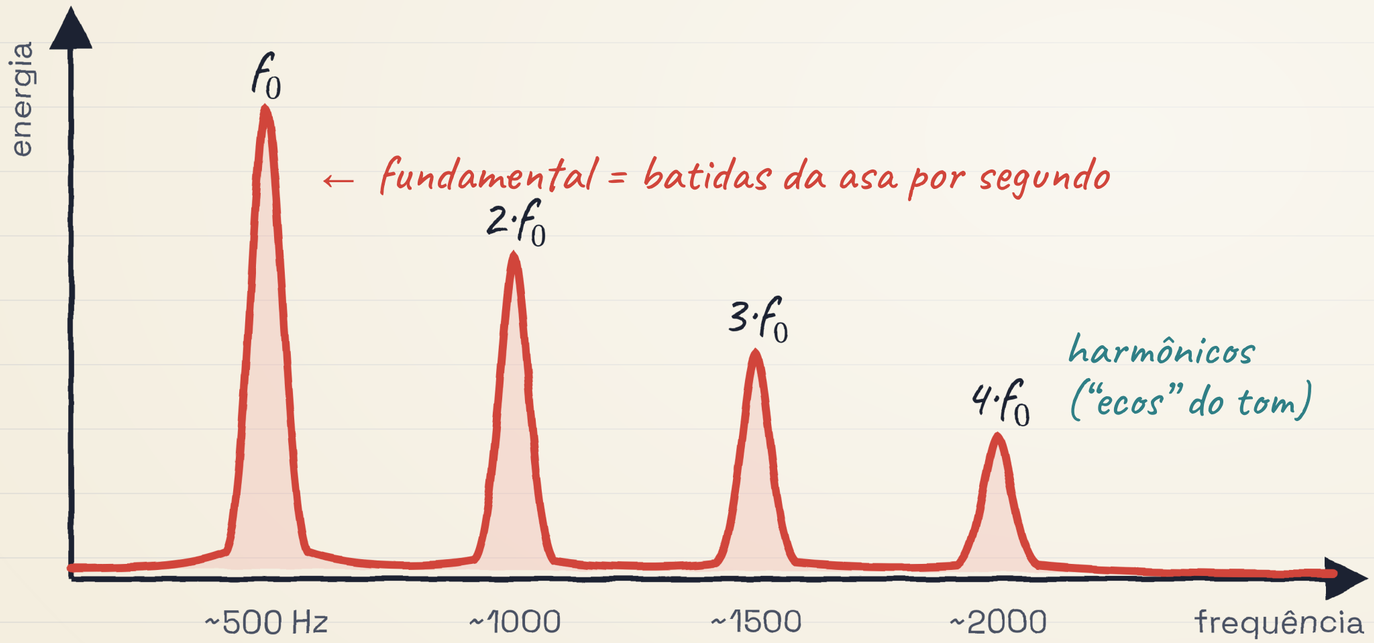
~500 batidas de asa por segundo

Cada espécie “canta”

O zumbido é o **batimento de asa**: centenas de Hz, com harmônicos. A ideia é reconhecer a espécie **pelo tom**.

a física do zumbido

O canto: um **tom** e seus **ecos**



como medimos f_0

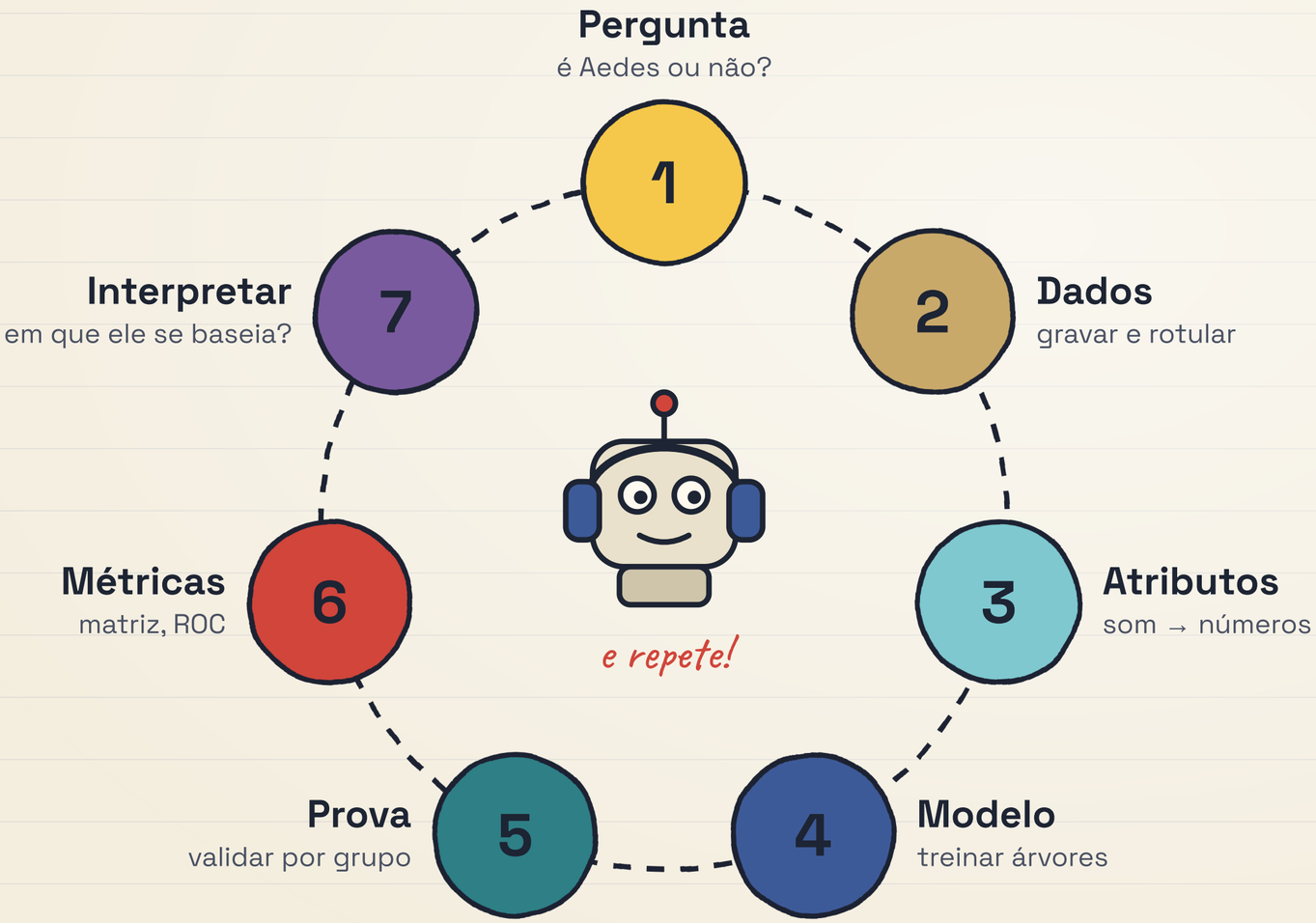
```python
# candidatos de 250 a 1000 Hz
cand = np.arange(250, 1001)
# soma harmônica (média dos logs)
harm = np.exp(sum(np.log(amp(k*cand))
          for k in range(1, 4)) / 3)
f0 = cand[np.argmax(harm)]  # o "tom"
```

⚠ Mas os tons **se sobrepõem:** Aedes aparece de ~300 a ~900 Hz, outras espécies de 250 a 1000 Hz. Macho ≠ fêmea. Só o tom não basta.

fonte: `reanalysis/frontend.py · descriptors()`

o trabalho de um dev de ML

Um projeto de ML é um ciclo



Na prática, a maior parte do tempo vai em dados e em provar que funciona — não em escolher o modelo.

Cada volta no ciclo responde a uma pergunta: **"o resultado é real ou o modelo está colando?"** Se a interpretação mostra algo estranho, voltamos aos dados.

é esse ciclo que vamos percorrer com o detector de mosquito

a caixa de ferramentas

As ferramentas do dia a dia

Python
A linguagem de quase todo projeto de ML. Legível e com bibliotecas para tudo.

NumPy · pandas
Tabelas e matrizes de números: cada gravação vira uma linha com 80 colunas.

SciPy · PyTorch
Processamento de sinais: filtros, reamostragem e o espectrograma mel.

scikit-learn
Os modelos (árvores), a validação cruzada e as métricas, prontos para usar.

Matplotlib
Gráficos: espectros, matriz de confusão, curva ROC. Olhar antes de acreditar.

VS Code · Jupyter
Editor e notebooks: código, texto e gráfico na mesma página.

Git · GitHub
Histórico de cada mudança: dá para voltar atrás e reproduzir um resultado.

Assistentes de IA
Claude Code e Codex: escrevem e rodam código comigo. (Volto nisso no fim.)

Arduino
E do lado da eletrônica: **Arduino Nano 33 BLE Sense** com microfone embutido, para gravar e testar o detector no mundo real.

os dados

Quem ensina o quê

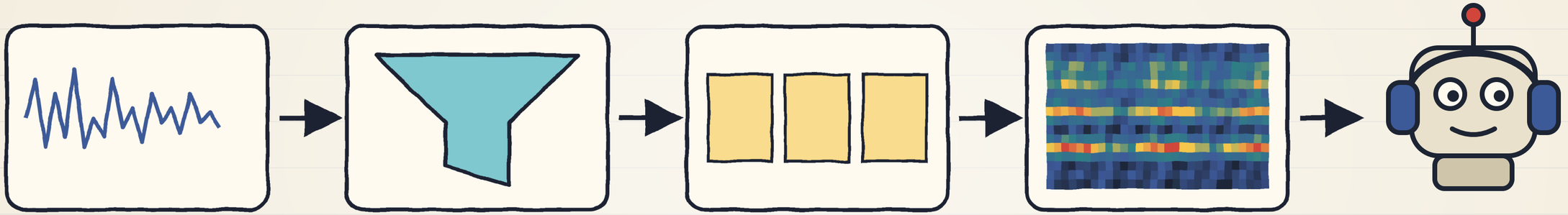
Um modelo de ML aprende **por exemplos rotulados**. A escolha dos exemplos é a primeira decisão de projeto:

1. **Aedes** de dois datasets grandes, **Torres** e **Funchal**: 2.167 exemplos.
2. Os outros datasets ensinam o que **não é Aedes**: 3.050 outros mosquitos e 426 ruídos.
3. Cada exemplo guarda **de onde veio** (fonte e aparelho). Isso vai servir para normalizar e para montar a prova.



como a máquina "ouve"

Do zumbido aos **números**



áudio bruto
.wav, .mp4, celular...

espectrograma mel
energia por faixa

16 bandas até 4 kHz
onde mora o zumbido

5 resumos × 16
= 80 números

modelo
ruído · outro · Aedes

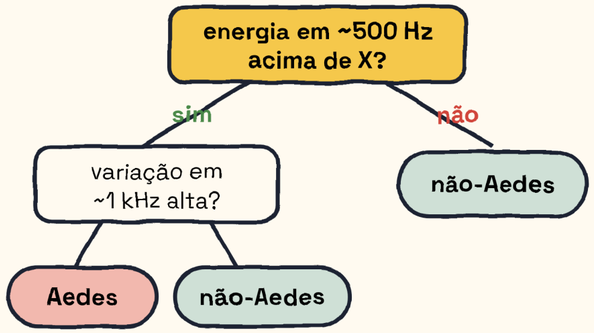
de espectrograma a 80 números

```
z = db[:, :, bandas_ate_4khz]      # mel em dB
z = z - z.max()                    # relativo ao pico
X = np.concatenate([z.mean(1), z.std(1),
     *np.quantile(z, [.1, .5, .9], axis=1)])
# 16 bandas × 5 resumos = 80 atributos
```

*A escala **mel** imita o ouvido. Para cada faixa, resumimos como a energia se comporta no tempo: **média**, **variação** e **percentis**. O batimento de asa (~400–900 Hz) cai em várias faixas.*

os modelos de ML

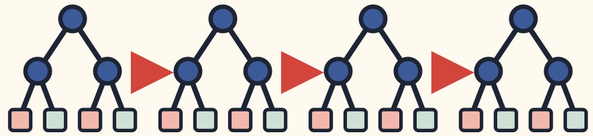
Árvores que fazem perguntas



Árvore de decisão

Uma sequência de perguntas sim/não sobre os 80 números. O treino escolhe **quais perguntas** e **quais limites** separam melhor as classes.

uma depois da outra, corrigindo erros

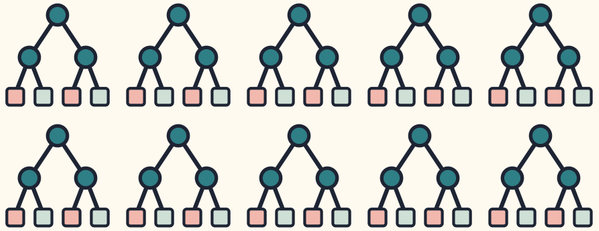


árvore 1 + 2 + 3 + … + 300

HistGB (boosting)

Árvores pequenas **em fila**: cada nova árvore foca nos **erros** das anteriores. A soma de muitas perguntas fracas vira um modelo forte.

todas ao mesmo tempo, depois votam



média de 400 votos

Extra Trees (floresta)

Centenas de árvores **independentes**, com cortes sorteados. Cada uma vota; a média é mais estável que qualquer árvore sozinha.

no scikit-learn

```
hgb = HistGradientBoostingClassifier(learning_rate=0.05, max_iter=300, max_leaf_nodes=15)
et  = ExtraTreesClassifier(n_estimators=400, min_samples_leaf=2)
p_aedes = (hgb.predict_proba(X)[:, 1] + et.predict_proba(X)[:, 1]) / 2   # média dos dois
```

a regra de ouro

Não deixar o modelo colar na prova

✗ dobras por arquivo



o mesmo celular aparece nos dois lados

✓ dobras por pote (cluster)



a prova é de um aparelho nunca visto

o "fiscal" da prova

```
sg = StratifiedGroupKFold(n_splits=5)
for tr, te in sg.split(X, y, groups=cluster):
    assert not (set(cluster[tr])
                & set(cluster[te])), \
        "cluster leakage"
```

Se o mesmo aparelho aparece no treino e na prova, o modelo pode **decorar o aparelho** e a nota fica inflada.

Grupo = gravação, indivíduo ou aparelho. **5 dobras**, repetidas **3 vezes** com sorteios diferentes.

como foi feito

Três ajustes no treino



cada aparelho tem seu "sotaque"

1 Normalizar por fonte

Cada fonte é centralizada nela mesma: some o "sotaque" do microfone, fica o mosquito.

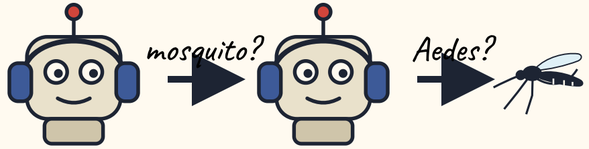
```
for fonte in fontes:
    m = origem == fonte
    X[m] = (X[m] - X[m].mean(0)) \
           / X[m].std(0)
```



2 Pesos por fonte

Uma fonte com 1.800 exemplos não pode "gritar" mais alto que uma com 20. Cada fonte pesa igual dentro da classe.

```
for fonte in fontes_da_classe:
    m = (y == c) & (origem == fonte)
    w[m] = 1 / m.sum() / n_fontes
modelo.fit(X, y, sample_weight=w)
```



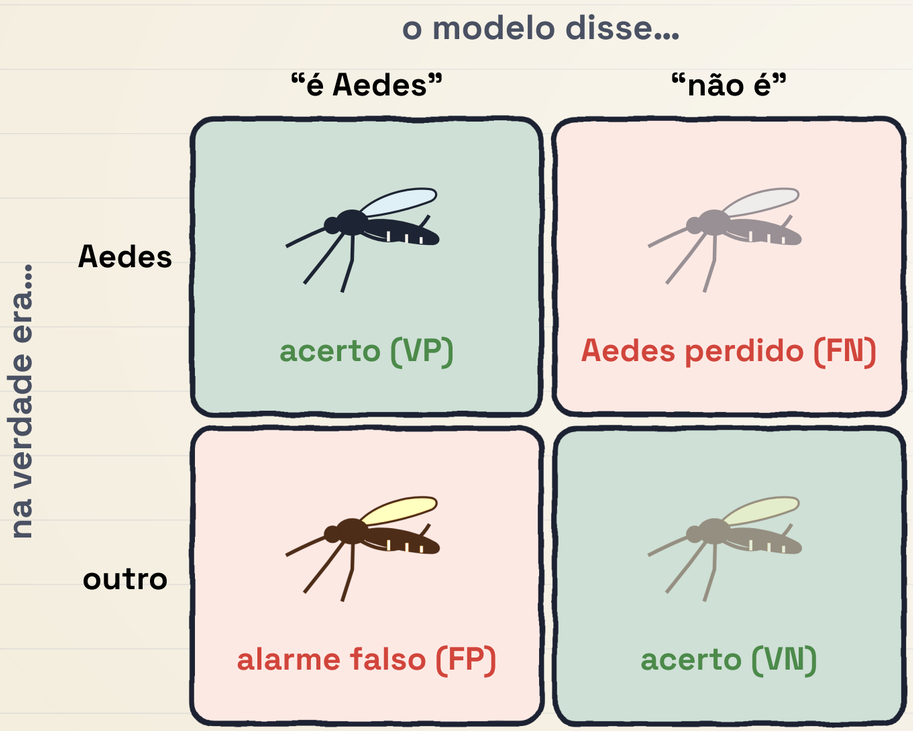
3 Dois estágios

Primeiro: **tem mosquito?** Depois, só entre mosquitos: **é Aedes?** (média de dois modelos de árvores).

```
p_m = etapa1.predict_proba(X)[:, 1]
p_a = (hgb.predict_proba(X)[:, 1]
     + et.predict_proba(X)[:, 1]) / 2
p = [1 - p_m,           # ruído
     p_m * (1 - p_a),   # outro
     p_m * p_a]         # Aedes
```

como medir

Acurácia não basta: três perguntas



a “matriz de confusão”

Precisão = VP / (VP + FP)

Quando o modelo apita “Aedes”, com que frequência acerta?

Detecção (recall) = VP / (VP + FN)

De todos os Aedes que existiam, quantos ele encontrou?

AUROC

Sorteie um Aedes e um não-Aedes: qual a chance de o Aedes ganhar nota maior? **0,5 = moeda**, 1 = perfeito. Não depende do limiar.

```
from sklearn.metrics import precision_score, recall_score, roc_auc_score
precision_score(y, y_pred); recall_score(y, y_pred)
roc_auc_score(y, p_aedes)   # usa a probabilidade, não o rótulo
```

com classes desbalanceadas, “acertar 90%” pode significar só chutar a classe mais comum

mão no código

Hora do VS Code

Um notebook curto que faz o ciclo inteiro: carrega os dados, treina, prova por grupo e desenha as curvas.

não precisa saber Python: leia os comentários e os gráficos

detector_aedes_demo.ipynb

```
[1] carregar os dados # 5.643 gravações × 80 atributos
[2] espectro médio por classe # um primeiro gráfico
[3] normalizar e pesar por fonte # tirar o “sotaque”
[4] treinar com prova por grupo # duas etapas, árvores
[5] matriz de confusão # onde acerta e onde erra
[6] curva ROC # AUROC
[7] escolher o limiar # o compromisso
[8] importância das frequências # interpretação
```

`apresentacao/demo/detector_aedes_demo.ipynb` · roda em ~1 min; os gráficos já ficam salvos no arquivo

o resultado, em uma tabela

A matriz de confusão do nosso modelo

era de verdade… \ o modelo disse…	ruído	outro mosquito	Aedes
ruído	**302** 71% da linha	**119** 28% da linha	**5** 1% da linha
outro mosquito	**234** 8% da linha	**2.546** 83% da linha	**270** 9% da linha
Aedes	**65** 3% da linha	**1.109** 51% da linha	**993** 46% da linha

Diagonal = acertos. 993 Aedes, 2.546 outros mosquitos e 302 ruídos.

O erro mais comum: 1.109 Aedes chamados de “outro mosquito”. O modelo é **cauteloso**.

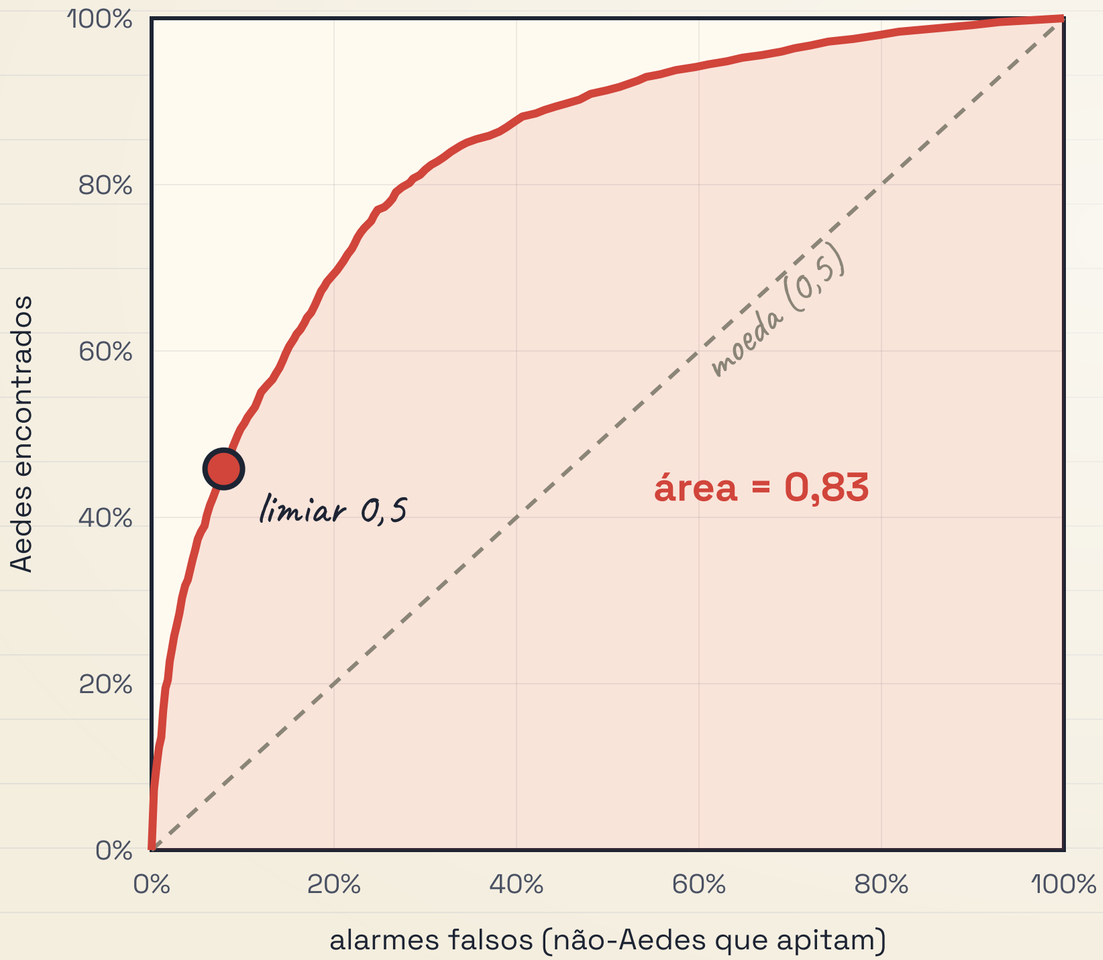
Alarmes falsos são raros: 270 de 3.050 outros mosquitos (9%) e só 5 de 426 ruídos.

Precisão = 993 / 1.268 = 78%
Detecção = 993 / 2.167 = 46%

5.643 gravações, cada uma prevista por um modelo que nunca a viu · limiar 0,5

curvas que ajudam a decidir

A curva ROC



Como ler

Cada ponto é um **limiar** diferente. Para cima = mais Aedes encontrados; para a direita = mais alarmes falsos. A diagonal é o chute (moeda).

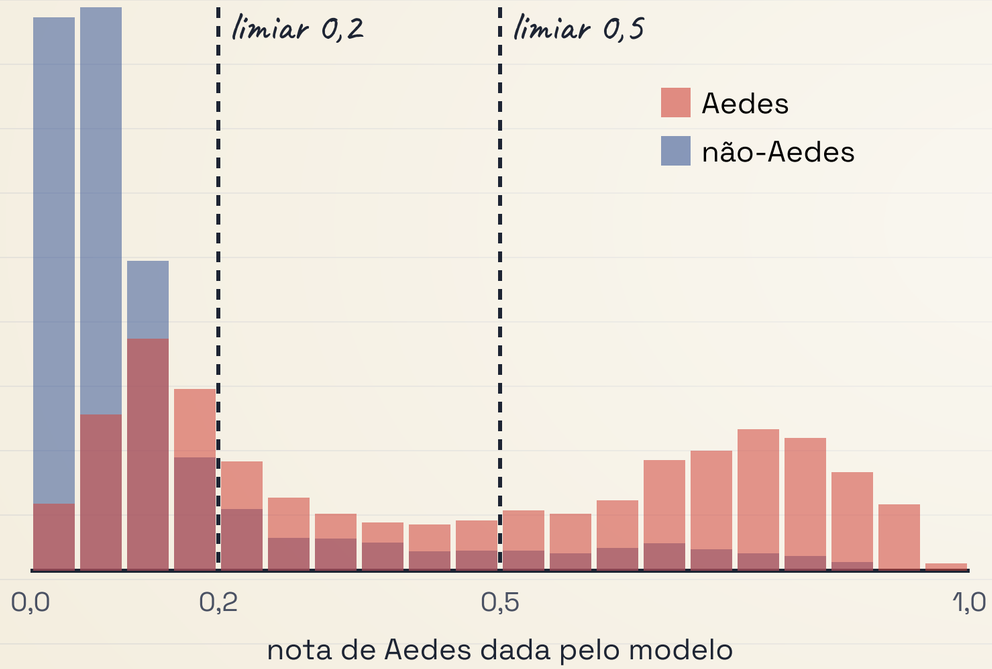
AUROC = 0,83

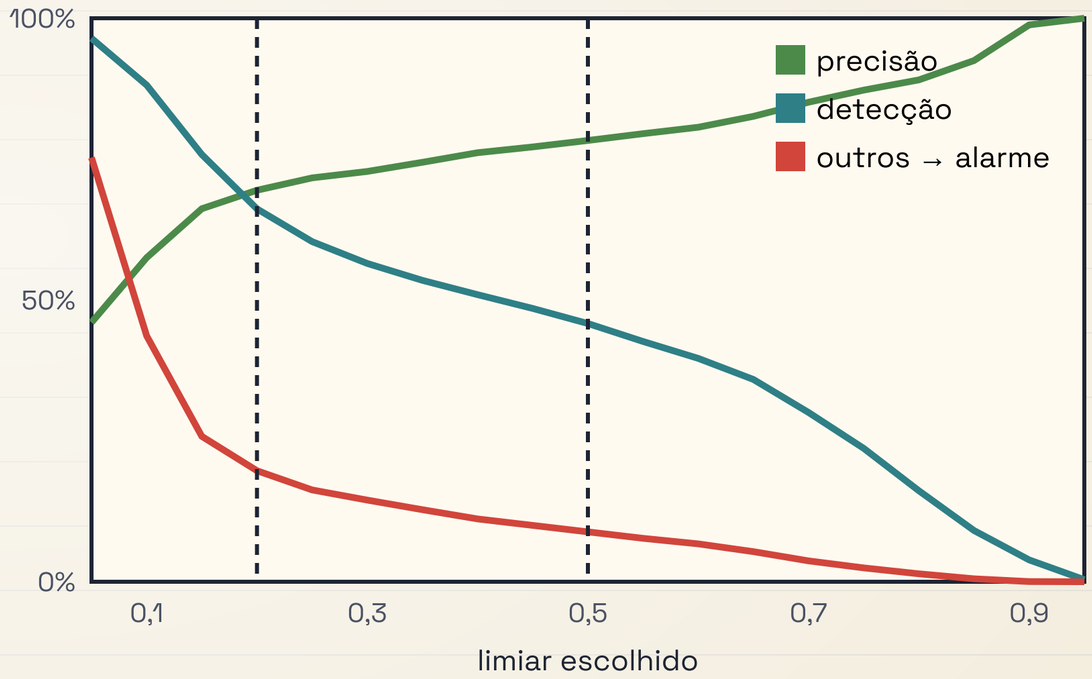
A área sob a curva. Escolhendo um Aedes e um não-Aedes ao acaso, em **83%** das vezes o Aedes recebe nota maior.

*O ponto vermelho é o limiar 0,5 que usamos: **46%** dos Aedes com só **8%** de alarmes falsos.*

uma decisão de projeto

Onde cortar a nota?





O modelo não diz "sim/não": dá uma nota de 0 a 1. Quem escolhe o corte somos nós.

Limiar 0,2 (vigilância, não perder Aedes): 66% detectados, 70% de precisão.

Limiar 0,5 (poucos alarmes): 46% detectados, 78% de precisão, 9% dos outros mosquitos apitam.

esquerda: notas que o modelo dá · direita: as três métricas para cada limiar

resultado

Quando apita, costuma ser Aedes

0,83
AUROC para Aedes

79%
dos alertas de Aedes estão certos

8%
dos outros mosquitos viram alarme

1%
dos ruídos viram alarme

De cada 100 alertas de “Aedes”...



79 são Aedes de verdade 21 são outro mosquito ou ruído

e só 8% dos outros mosquitos (1% dos ruídos) disparam alarme

*Quando o modelo diz “é Aedes”, ele acerta **quase 8 em cada 10** vezes.*

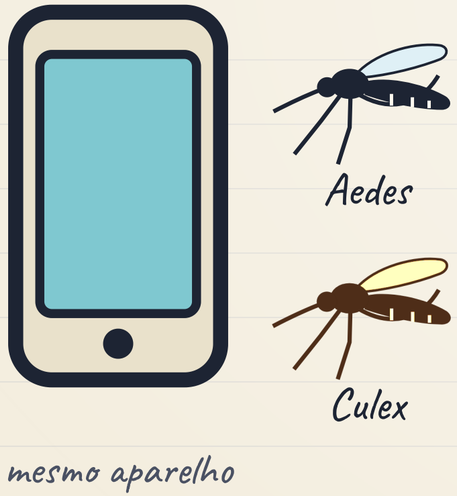
O modelo é **cauteloso**: prefere ficar quieto a dar alarme falso. Detecta **46%** dos Aedes, mas quando apita, costuma estar certo.

média de 3 repetições · validação cruzada por grupo

a prova mais justa

Aedes × *Culex* no mesmo aparelho

Aedes e *Culex* gravados nos **mesmos aparelhos.** Deixamos um aparelho inteiro fora do treino e perguntamos: o modelo separa as duas espécies?





0,73

AUROC Aedes × Culex · aparelho nunca visto

Por que essa prova vale mais

O aparelho é o mesmo para as duas espécies, então **o "sotaque" do microfone não ajuda.** Para acertar, o modelo precisa ouvir o mosquito.

Ainda falta

O modelo só viu Aedes de **dois datasets.** Aedes gravados em **outros aparelhos** ainda não são reconhecidos: o próximo passo é coletar mais fontes.

interpretabilidade

O que o modelo está **ouvindo**?

As métricas dizem **quanto** o modelo acerta.
A interpretabilidade pergunta **em que ele se baseia**.

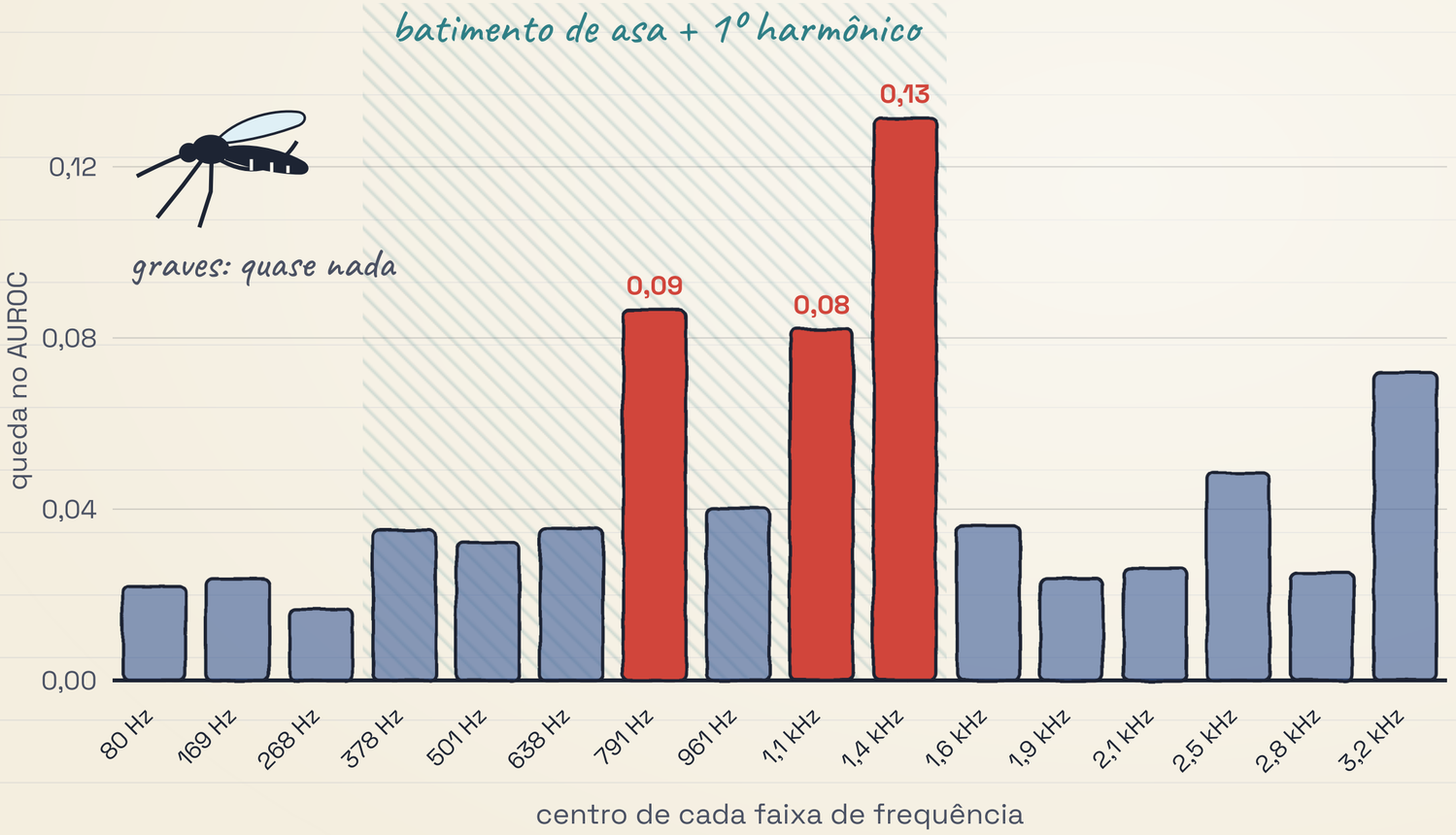
🎧 quais frequências ele usa?

🔍 ele ainda “ouve” o aparelho?



importância por permutação

Ele presta atenção no batimento de asa



embaralhar uma faixa

```
for banda in range(16):
    Z = X_teste.copy()
    c = colunas(banda)  # 5 resumos
    Z[:, c] = Z[sorteio][:, c]
    queda[banda] = auroc_base \
        - roc_auc_score(y, modelo(Z))
```

Se embaralhar uma faixa **derruba** o AUROC, o modelo depende dela. As maiores quedas estão em **0,6–1,6 kHz**: o batimento e seu primeiro harmônico. Os graves (< 300 Hz) quase não pesam.

E o aparelho? Depois da normalização, um classificador linear não consegue mais dizer de qual fonte veio um mosquito: **17%** (acaso: 20%).

IA ajudando a fazer IA

Como usei assistentes de IA no código

1 Eu descrevo
o objetivo, em português

2 A IA escreve
e roda o código

3 Eu confiro
números, gráficos, testes

4 Ajustamos
e repetimos

Onde ajudou muito

Escrever código repetitivo, montar gráficos, ler arquivos de vários datasets, testar várias ideias rápido, revisar e documentar.

Onde precisei conferir

Se o treino e a prova estavam mesmo separados; se os arquivos foram lidos certo (um erro na leitura das pastas só apareceu porque o programa quebrou e fomos investigar); se o número fazia sentido.

A regra

A IA acelera, mas **a responsabilidade pelo resultado é sua.** Peça para ela explicar o código e rode testes que poderiam dar errado.

eu: "Junte os datasets, use Aedes só do Torres e do Funchal, valide por aparelho e me mostre a matriz de confusão."

```
# o teste que eu sempre peço junto:
assert not set(grupo[tr]) & set(grupo[te]), "vazamento!"
```

Do modelo ao hardware



o protótipo: a placa dentro de um pote-armadilha

tudo dentro de uma placa do tamanho de um dedo



o mesmo ciclo de ML, só que com pouca memória e pouca energia

no Arduino (C++)

```
// interrupção do microfone PDM:
void onPDMdata() {
  int n = PDM.read(chunk, PDM.available());
  // guarda no buffer circular
  // → atributos → modelo (.h) → nota
}
```

Arduino Nano 33 BLE Sense: microfone MEMS digital, processador ARM Cortex-M4 de 64 MHz e 256 KB de RAM.

Hoje: firmware **experimental** de presença de mosquito. O detector de espécie roda no computador.

próximos passos

Receita para um teste de verdade

- [ ] **Controles pareados:** Aedes *e* outras espécies gravados no **mesmo aparelho, setup e dia** — em vários aparelhos e locais.
- [ ] **Ruído in situ** dos ambientes reais, no lugar de ESC-50/Pete.
- [ ] **Mais fontes de Aedes:** o modelo só viu Aedes de dois datasets e ainda não reconhece Aedes de aparelhos novos.
- [ ] **Um teste cego único**, numa campanha nova, com o protocolo congelado **antes** da coleta.



conclusão

O que aprendemos ouvindo mosquitos

1

Dados primeiro: escolher o que cada dataset ensina e normalizar por fonte pesou mais que trocar de modelo.

2

Prova honesta: validar por grupo e aparelho. Resultado: **79% de precisão**, AUROC **0,83**; Aedes × Culex no mesmo aparelho: **0,73**.

3

Interpretabilidade: o modelo se apoia na faixa do **batimento de asa (0,6–1,6 kHz)**, não no aparelho.

Obrigado!

perguntas? bzzz...



Luiz Eduardo do Nascimento Ciarallo · `mosquito-wingbeat`